



BACKEND IMPLEMENTATION GUIDE

AI-Based Behavioral Fraud Detection System

Technical Development Handbook



CURRENT PROJECT STATE



Existing Repository Components

Component	Status	Notes
 PostgreSQL Schema	 Complete	<code>db/schema.sql</code>
 Database Business Logic	 Complete	<code>db/functions.sql</code>
 Seed Data	 Complete	<code>db/seed.sql</code>
 Docker Setup	 Complete	<code>docker/docker-compose.yml</code>
 Backend Code	 Mostly Empty	<code>app.js</code> , <code>server.js</code> , <code>.env</code>
 Dependencies	 Partial	Missing <code>axios</code> , <code>express-rate-limit</code>



IMPORTANT IMPLEMENTATION NOTE

Source of Truth

Use:

`db/`

Do **NOT** follow Prisma examples from documentation.

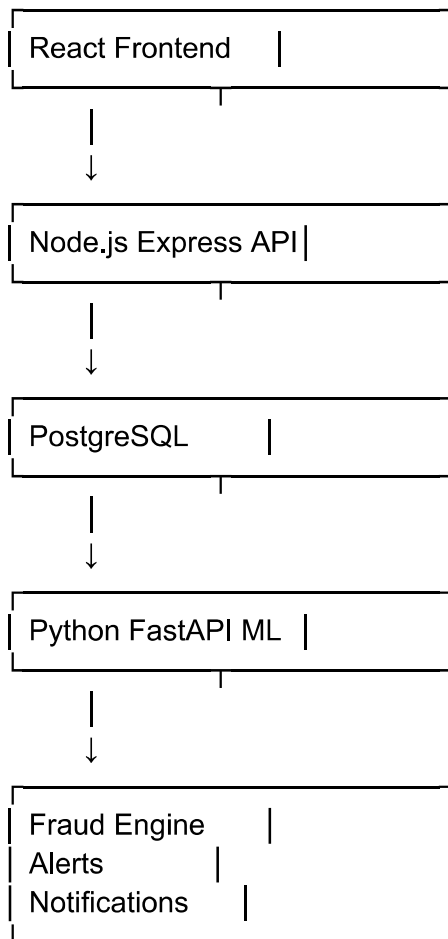
Current repository already uses:

- ✓ Raw SQL
- ✓ PostgreSQL Functions
- ✓ Existing DB logic

Avoid replacing business logic with ORM implementations.



SYSTEM ARCHITECTURE





PHASE 0 — ENVIRONMENT + DATABASE SETUP



Goal

Working PostgreSQL database with schema and seed data.



Required Installation

Install:

✓ Node.js

✓ Git

✓ PostgreSQL

✓ Docker

✓ Postman



Start Database

Run:

`docker compose up`

► Execute SQL Files

Execute in exact order:

`db/schema.sql`

`db/functions.sql`

db/triggers.sql

db/seed.sql

Verify Database

Run:

```
SELECT * FROM users;
```

```
SELECT * FROM accounts;
```

Completion Checklist

- ☒ Database container running
 - ☒ Tables created
 - ☒ Seed data inserted
 - ☒ User/account records visible
-



PHASE 1 — EXPRESS FOUNDATION



Goal

Working Express API skeleton.



Folder Structure

backend/

└─ src/

|— config/

|— routes/

|— controllers/

|— services/

|— middleware/

|— utils/

|— app.js

|— server.js

.env

package.json



Install Dependencies

npm install express cors helmet morgan dotenv pg cookie-parser nodemon axios
express-rate-limit

⚙️ app.js Setup

Add middleware:

express()

cors()

helmet()

morgan()

cookieParser()

express.json()

Health Route

Route:

GET /health

Response:

```
{  
  "status": "OK"  
}
```

server.js Responsibilities

- ✓ Load dotenv
 - ✓ Initialize Express app
 - ✓ Start listening on port
-

Environment Variables

PORT=

DATABASE_URL=

JWT_SECRET=

ML_API_URL=

CORS_ORIGIN=

✓ Completion Checklist

- ☑ Server starts successfully
 - ☑ Middleware loads
 - ☑ Health route returns 200
-



PHASE 2 — DATABASE CONNECTION LAYER



Goal

Reusable DB access layer.



Create Files

◆ `src/config/db.js`

Responsibilities:

Create PostgreSQL Pool

Export DB connection

◆ `src/utils/asyncHandler.js`

Responsibilities:

Handle async route errors

◆ `src/middleware/errorHandler.js`

Responsibilities:

Handle:

- DB errors
- Validation errors
- JWT errors
- ML timeout errors

Table Mapping

Domain	Database Tables
 Users	users
 Accounts	accounts
 Transactions	transactions
 Beneficiaries	beneficiaries
 Sessions	logins, known_devices
 Fraud	telemetry, fraud_scores

✓ Completion Checklist

- ☒ Database connection established
- ☒ SELECT test works
- ☒ DB reusable across services

PHASE 3 — AUTHENTICATION

Goal

JWT-based authentication.

APIs

Register

POST /api/auth/register

Login

POST /api/auth/login

Profile

GET /api/auth/profile

Logout

POST /api/auth/logout

Service Responsibilities

Register Flow

User Input



Validate PAN/Form60



Hash password



Insert user



Create account

Login Flow

User credentials



Compare password



Generate JWT



Return token

JWT Payload

```
{  
  "user_id":1,  
  "acc_id":101  
}
```

✓ Completion Checklist

- ☒ Registration works
 - ☒ Login works
 - ☒ JWT generated
 - ☒ Protected routes work
-



PHASE 4 — LOGIN TRACKING + DEVICE INTELLIGENCE



Goal

Track login risk signals.

Store Data



IP Address



Device fingerprint



Browser



User Agent



Location



VPN Probability



APIs

GET /api/sessions

POST /api/devices/register



Detection Rules



Failed Login Detection

Failed Attempts > Threshold



Update telemetry



Mark suspicious



New Device Detection

Current fingerprint

VS

Stored fingerprints



Impossible Travel Detection

Current Login Location



Previous Location



Time Difference Analysis



VPN Detection

Store:

VPN_probability

✓ Completion Checklist

☒ Login records inserted

☒ Device records inserted

- ☑ Failed login tracking works
 - ☑ Suspicious sessions generated
-

PHASE 5 — CORE BANKING APIs

Goal

Money movement through SQL functions.

Account APIs

GET /api/accounts/me

GET /api/accounts/:acctId/balance

Beneficiary APIs

GET /api/beneficiaries

POST /api/beneficiaries

Transaction APIs

POST /api/transactions/p2p

POST /api/transactions/merchant

GET /api/transactions

GET /api/transactions/:id

⚙ Database Functions

Use existing SQL functions only:

SELECT transfer_money()

SELECT merchant_payment()

SELECT add_beneficiary()

Validation Rules

- ✓ Sender owns account
 - ✓ Account ACTIVE
 - ✓ Balance available
 - ✓ Amount $\leq$ transfer limit
 - ✓ No self-transfer
-

✓ Completion Checklist

- ☒ P2P transfers work
 - ☒ Merchant payments work
 - ☒ Beneficiary creation works
 - ☒ Transaction history works
-



PHASE 6 — BEHAVIORAL TELEMETRY



Goal

Generate ML input features.



Capture

 Mouse speed

 Typing speed

 Click frequency

 Copy-paste count

Compute

amount_to_balance_percentage

amount_to_limit_percentage

frequency_of_transactions

deviation_from_avg_amount

no_of_prior_transfers



API

POST /api/telemetry

✓ Completion Checklist

☒ Telemetry records inserted

☒ Feature computation works



PHASE 7 — ML INTEGRATION



Goal

Connect Node backend to FastAPI model.



ML Endpoint

POST /predict

Expected Response

```
{
  "fraudProbability":0.91,
  "risk_score":91,
  "riskLevel":"HIGH"
}
```

Risk Mapping

● 0–30 → LOW

● 31–70 → MEDIUM

● 71–100 → HIGH



High Risk Actions

High Score



Block Transaction



Create Alert



Notify User



Optional Account Block

✓ Completion Checklist

- ☒ ML request works
 - ☒ fraud_scores records created
 - ☒ Blocking logic works
-



PHASE 8 — ALERTS + NOTIFICATIONS



APIs

GET /api/alerts

GET /api/alerts/:id

PATCH /api/alerts/:id/resolve

POST /api/alerts

GET /api/fraud-scores

POST /api/notifications

GET /api/suspicious-sessions

Alert Flow

ML Score HIGH



`create_fraud_alert()`



notification_logs



Send Email/SMS/Push



Account Hold (optional)

✓ Completion Checklist

- ☒ Alerts generated
 - ☒ Notifications generated
 - ☒ Alert resolution works
-



PHASE 9 — SECURITY + HARDENING

Add Security Layers

- ☒ Helmet
- ☒ CORS restrictions

- ☑ Rate limiting
 - ☑ Input validation
 - ☑ Parameterized SQL
 - ☑ JWT expiration handling
 - ☑ Database timeout handling
 - ☑ ML timeout handling
-

Never Return

pass_hash

PAN_hash

raw JWT token data

Completion Checklist

- ☑ Invalid payload → 400
 - ☑ Brute-force protection works
 - ☑ Secure API responses
-

PHASE 10 — TESTING + DEMO READINESS

Postman Scenarios

Scenario 1

Login

↓

Small transfer

↓

Success

Scenario 2

High-value transaction

↓

ML detects risk

↓

Blocked transaction

Scenario 3

VPN Login

↓

Suspicious Session

↓

Alert Generated

Scenario 4

Suspicious Merchant

↓

Blocked payment

Scenario 5

New Device Login



Risk score generated



DEVELOPMENT TIMELINE

Day 1 → Phase 0–1

Day 2 → Phase 2–3

Day 3 → Phase 4–5

Day 4 → Phase 6–7

Day 5 → Phase 8

Day 6 → Phase 9–10

Day 7 → Docker + Demo Polish



IMPLEMENTATION PRIORITY ORDER

1 Authentication

2 Transactions

3 ML Communication

4 Fraud Scores

5 Dashboard APIs

FINAL FLOW

Login



Device Detection



Telemetry Collection



Transaction



ML Prediction



Fraud Score



Alert Generation



Dashboard Update
